## Experiment 7: Minimax Algorithm

### Aim:

To implement the Minimax algorithm for selecting the optimal move in an adversarial two-player game.

### Algorithm:

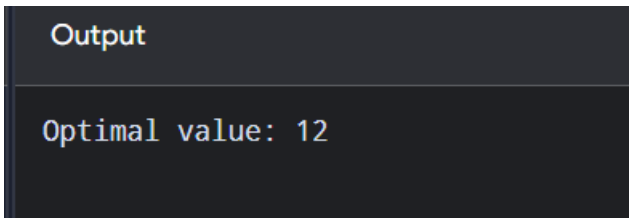
1. Maximizer tries to maximize the score; Minimizer tries to minimize it.
2. Recursively explore the game tree to terminal nodes.
3. At maximizer node: choose child with maximum value.
4. At minimizer node: choose child with minimum value.
5. Back up values to the root to select the best move.

### Code:

```
def minimax(depth, nodeIndex, maximizing, values, maxDepth):
    if depth == maxDepth:
        return values[nodeIndex]
    if maximizing:
        best = float('-inf')
        for i in range(2):
            val = minimax(depth+1, nodeIndex*2+i, False, values, maxDepth)
            best = max(best, val)
        return best
    else:
        best = float('inf')
        for i in range(2):
            val = minimax(depth+1, nodeIndex*2+i, True, values, maxDepth)
            best = min(best, val)
        return best

values = [3, 5, 2, 9, 12, 5, 23, 23]
print("Optimal value:", minimax(0, 0, True, values, 3))
```

### Sample Output:




### Result:

Minimax correctly evaluated the game tree and determined the optimal value of 12 for the maximizing player.